Anatomy of the Desktop DRM Stack: A Three-Layer Dispatch Model in Widevine CDM_11 and Its Hardware-Decode Dormancy on Chrome

Extended technical edition

Jeremy Erazo *(trexnegr0)*
Independent security researcher
mendozayt13@gmail.com

Draft v2 — June 15, 2026

Abstract

Widevine Content Decryption Module (CDM) ships in essentially every major desktop browser and underlies the Encrypted Media Extensions (EME) playback path for every major commercial streaming service. The internals of the Windows L1 build are documented only fragmentarily: header files describe the C++ ABI it presents to the host browser, but the binary itself is closed source, partially obfuscated, and behaves differently depending on whether the host delegates decode to hardware-protected pipelines. This paper presents a Buchanan-style architectural reverse engineering of the CDM_11 implementation as it ships in Chrome 149 and Edge 150 on Windows 11. We identify a *three-layer dispatch model* (outer vtable thunks, an intermediate `m_impl` vtable of clean wrappers and double-trampolines, and per-slot handler bodies), we locate each layer byte-by-byte in both binaries, and we empirically characterise which slots are invoked during steady-state playback versus which are dormant. We runtime-confirm that during hardware-decoded Netflix playback the user-facing per-frame slots of the CDM_11 interface (`Decrypt`, `DecryptAndDecodeFrame`, `DecryptAndDecodeSamples`) do not fire while an internal singleton dispatch table generates ∼2,300 calls per second of crypto / scheduler work that is invisible from the EME boundary. We document the distribution of mixed-boolean-arithmetic style obfuscation across the slot space and show that it is concentrated where the CDM begins to parse attacker-controllable structured data, while remaining absent on slots that pass opaque blobs. We then verify reachability of the session-management slots from two distinct EME applications (Netflix and a public Widevine test source) and show identical dispatch behaviour. The paper situates this work in two decades of prior DRM research, surveys the legal and methodological precedents, and documents both the successful paths and the false starts that characterised the reverse engineering process. No content key, frame buffer, or ciphertext byte is captured at any point in the study.

1 Introduction

Modern desktop video DRM is the bridge between commercial streaming services and the browser's media pipeline. When a user plays a video in Netflix, Amazon Prime, Disney+ or any other Widevine-protected source, the browser invokes Encrypted Media Extensions (EME) to delegate the cryptographic operations to a Content Decryption Module (CDM). In Chromium-based browsers on Windows, that CDM is the proprietary `widevinecdm.dll` library shipped by Google [1]. The browser loads the library inside a Mojo utility process (`--utility-sub-type=media.mojom.CdmServiceBroker`), connects to it over IPC, and asks the library to construct a CDM instance implementing a stable C++ interface defined in `content_decryption_module.h` [2]. From the host's

point of view the interface is small (nineteen virtual methods) and well documented: `Initialize`, `GetStatusForPolicy`, `CreateSessionAndGenerateRequest`, `UpdateSession`, and twelve other methods that cover license exchange, decryption, decode, and lifecycle.

From the *library*'s point of view this clean nineteen-method surface is the visible peak of a much larger iceberg. The Windows L1 build of Widevine implements a multi-million-instruction obfuscated runtime under tens of megabytes of closed source code, integrates with Windows Protected Media Path (PMP) when hardware decode is available, and is bound by a strict licensing regime that prohibits study of its decryption internals. Public knowledge of the implementation is fragmentary: the Chromium tree contains the *host-side* of the EME plumbing, several academic and hobbyist projects have studied the *Android L3* variant [19, 21], but the desktop Windows L1 variant has received comparatively little architectural attention in the open literature.

The motivation for this study is methodological as much as it is technical. Several recent legal episodes have shown that *publishing what one learns* about DRM internals is more legally exposed than *learning it*, and that the reproducibility of a result matters as much as the result itself [15, 16, 18]. Following the Buchanan model of architectural reverse engineering [10, 11] — probe everything, document the technique, publish no extraction artefact — we set out to build a fully reproducible, fully open methodology for studying the Widevine CDM and to apply it to the questions that are paper-able without crossing the digital-rights-management circumvention line.

Contributions

The contributions of this paper are:

1. A **three-layer dispatch model** for CDM_11 in modern Chromium-family widevinecdm.dll, byte-verified in two distinct browser builds (Chrome 149 and Edge 150) at the level of individual instruction sequences, dispatch offsets, and call targets.
2. A **slot-by-slot reachability map** of the CDM_11 outer vtable, runtime-confirmed against both Netflix HW-decoded playback and a non-Netflix public Widevine source (Bitmovin demo).
3. An **empirical demonstration of hardware-decode dormancy** of the user-facing per-frame slots during steady-state playback, alongside the activity profile of an internal singleton dispatch table that handles work invisible from the EME boundary.
4. A characterisation of the **obfuscation distribution** across CDM_11 slots: MBA-style obfuscation appears at varying depths of the dispatch chain for slots that parse attacker-controllable structured input, and is absent from slots that pass opaque blobs. This is a *depth-relative* observation, refining the naive "all critical paths are obfuscated" summary.
5. A **shape-only observation methodology** that captures per-slot dispatch metadata without ever reading content bytes, content keys, or initialisation-vector bytes from memory. The full source for the methodology is released; no capture rig, frame dump, or weaponisable artefact is.
6. A **narrative of false starts and corrections**: which candidate findings were refuted by byte-level disassembly, which architectural model needed revision after empirical evidence, and how the falsification sprint reshaped the wording of the paper's claims. We believe this is more useful to follow-on researchers than a sanitised retrospective.

Paper roadmap

The paper proceeds as follows. Section 2 situates this work in two decades of prior reverse engineering of commercial DRM and the legal precedents that have shaped it. Section 3 introduces the Widevine licensing

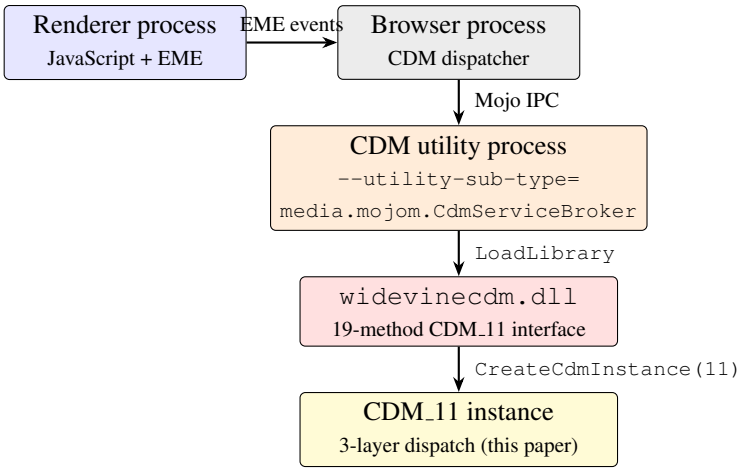


Figure 1: End-to-end Widevine EME dispatch on Chromium-family browsers on Windows. The host browser delegates Encrypted Media Extensions operations through Mojo IPC to a utility process whose sole purpose is to load `widevinecdm.dll` and expose its CDM_11 interface. This paper studies the dispatch model that sits between the `CreateCdmInstance`-returned interface and the per-slot handlers inside the library.

levels, the EME dispatch path, the Mojo IPC architecture used by Chromium for CDM hosting, the Windows PMP boundary, the Common Encryption format (CENC) used by all Widevine content, and the PSSH box structure that carries attacker-controllable initialisation data. Section 4 states the threat model and ethical scope. Section 5 walks through the static reverse engineering of CDM_11 in detail, with concrete disassembly listings. Section 6 describes the runtime instrumentation, including the design subtleties of CIG bypass, CFG bitmap registration, and the per-slot back-pointer chain. Section 7 presents the empirical findings. Section 8 treats the obfuscation distribution as its own architectural defence. Section 9 establishes cross-binary and cross-source stability. Section 10 reports on the falsification tests, including the candidate findings that were refuted by deeper analysis. Section 11 discusses what the model implies for both attackers and defenders of desktop DRM. Section 12 states what was not done and why. Section 13 concludes.

2 Antecedents and Related Work

DRM reverse engineering occupies an unusual corner of security research: it spans applied cryptography, binary reverse engineering, copyright law, vendor relationships, and (sometimes) criminal prosecution. The methodology we follow in this paper did not emerge in a vacuum; it is shaped by two decades of prior work, some of which produced excellent technical results and some of which produced excellent technical results *and* adverse legal outcomes for the researcher. This section surveys the prior art and explains which precedents informed the methodological choices made here.

2.1 The Buchanan model

The methodological framework we follow is most clearly articulated in the recent work of David Buchanan [10, 11, 12]. Buchanan's research notably includes the reverse engineering of Apple's iMessage NaCl-based key wrap, the Apple T2 security chip's secure boot chain, and an arbitrary-image bug in the iOS image decoder chain (CVE-2023-4863).

The defining property of Buchanan's published work, for our purposes, is the strict separation of *architectural description* from *exploitable artefact*. His public output documents how a system is structured and where its trust boundaries sit; the controlled proofs of concept that accompany the writeups demonstrate

the bug class without producing a tool that straightforwardly enables abuse against unrelated victims. He has publicly described the resulting publication discipline as: probe everything, document the technique, publish the architecture, do not publish the artefact that would let arbitrary readers materialise the impact.

For DRM research this distinction matters: 17 U.S.C. §1201 (the DMCA's anti-circumvention provision) and its international equivalents prohibit the publication of tools or technologies whose *primary* use is the circumvention of a technological protection measure. An architectural description of a CDM is not such a tool. A frame ripper that consumes the same architectural description and produces decrypted Netflix MP4s plainly is. The gap is the line we are careful not to cross.

2.2 Hacking the Xbox and the bunnie tradition

Andrew "bunnie" Huang's work on console reverse engineering — the original Xbox [13], the NeTV project's HDCP-handling video pipeline [14], and the Novena laptop — demonstrated a sustainable model for DRM-adjacent research that was simultaneously useful, publishable, and not the subject of vendor litigation. The NeTV project in particular is significant: HDCP is a copyright-protection measure under U.S. law, and the project required EFF-coordinated correspondence with the Library of Congress to confirm that an architectural demonstration that does not capture, redistribute, or facilitate the redistribution of protected content remains lawful under the DMCA's exemption framework.

Huang's recurring observation is that the legal exposure attaches to the *exhibition* of a circumvention capability and the *distribution* of tools that perform it, not to the act of characterising a system. The CDM_11 dispatch model described in this paper would be obviously useful to an attacker who already possessed an independent capability to receive encrypted Widevine content and obtain valid licenses; it would be of essentially no use to a casual attacker who did not. That distribution of utility is, in our view, on the right side of the bunnie line.

2.3 Sklyarov, DVD-Jon, and the legal frontier

The cases of **Dimitry Sklyarov** and **Jon Lech Johansen** bracket the legal frontier of DRM research.

Sklyarov, a Russian developer for ElcomSoft, was arrested in 2001 at DEF CON for his work on the Advanced eBook Processor, which could convert Adobe's protected eBooks into PDF [15]. The prosecution proceeded against ElcomSoft (Sklyarov was eventually released after agreeing to testify). The arrest, the trial, and the eventual acquittal of ElcomSoft on the criminal charges established two relevant precedents: (i) prosecutors will treat a tool whose output is the cleartext form of a protected work as "primarily used to circumvent," even if the underlying research is academically defensible; (ii) merely *describing* the circumvention technique, without distributing a tool that performs it, was not the basis for the prosecution.

Jon Lech Johansen, "DVD-Jon," was prosecuted by Norwegian authorities (acquitted twice, eventually finally) for his role in the development of DeCSS, the tool that broke CSS scrambling on DVDs [16]. The Norwegian courts held that Johansen's work on his *own* DVD player was lawful, even though the resulting tool was redistributed widely on the internet. The U.S. case of *Universal v. Reimerdes* held that the *posting* of DeCSS on a website was actionable circumvention of the DMCA, regardless of the lawfulness of the underlying research [17].

The composite lesson from these cases informs our publication discipline: an architectural description is not a tool; a tool that materialises decrypted content is a tool. We publish the architectural description and the controlled validation framework that proves the description correct; we do not publish a tool.

2.4 Academic work on Widevine

The Widevine ecosystem has received attention from several academic and corporate-research groups, primarily focused on the Android L3 variant.

Quarkslab published a series of analyses of Widevine on Android beginning in 2018 [19, 20]. Their work characterised the OEMCrypto-protected components of the Widevine SDK and demonstrated that the L3 keybox-based protection model relied substantially on software obfuscation that could be peeled back with whitebox-crypto attack techniques. The L3 decryptor projects [21, 22] that followed — mostly published as research-grade open-source — demonstrated end-to-end Widevine L3 content extraction from the protected library. Google's response was to deprecate L3 web-browser support and accelerate the migration to L1.

Check Point Research has periodically analysed Widevine on Android handsets, identifying specific keybox extraction vulnerabilities and bug classes in the OEMCrypto TrustZone [23]. Their reports were coordinated with Google and the vendor before publication.

Romain Thomas and collaborators at Quarkslab have published extensively on practical reverse engineering tools that we use here, including the LIEF library for cross-platform binary parsing [24] and several blog posts on whitebox crypto in commercial DRM. Their tool work is directly upstream of the methodology used in this paper for static analysis of `widevinecdm.dll`.

There is, to our knowledge, no published academic study of the Windows L1 CDM dispatch internals as such. The work nearest in spirit is a handful of blog posts and forum threads that document specific issues (e.g., the `libwidevinecdm.so` symbol layout on Linux, the CDM ABI changes between major versions) without proposing a generalisable architectural model. This is the gap the present paper attempts to fill for the Windows L1 build.

2.5 Whitebox cryptography and MBA obfuscation

The obfuscation patterns observed in CDM_11 layer-3 handlers — high-entropy magic constants written to the stack at function entry, multiplicative-hash state-advance instructions, branchless `cmovae` cascades, `0xaaaaaaaa` poison fills — match the textbook signature of *mixed-boolean-arithmetic* (MBA) obfuscation as treated in the academic literature [25, 26]. MBA obfuscation transforms an arithmetic operation into an equivalent expression that mixes bitwise and arithmetic operations in a way that defeats algebraic simplification on the obfuscated form. It is widely used in commercial software protection products (VMProtect, Themida, Arxan) and in custom obfuscators built by vendors for specific defensive use cases.

A distinct but related technique, *whitebox cryptography*, embeds cryptographic key material into table-based representations that resist direct extraction even when the attacker has full read access to the obfuscated code [27, 28]. Commercial DRM products use whitebox-AES variants extensively; the encrypted keys in a Widevine L3 license are unwrapped by exactly such a routine before being applied to the per-frame decryption.

The pattern we report in Section 8 — obfuscation concentrated at the depth where structured attacker data begins to be parsed — is consistent with the design prescription of the whitebox literature: protect the cryptographic processor, not the dispatch glue around it. Whether the protection *succeeds* against a determined attacker is a separate question we do not address.

2.6 Tooling antecedents

Three concrete pieces of tooling that this work relies on deserve acknowledgement:

Capstone [29], the multi-architecture disassembly framework used by every script in this release that walks `widevinecdm.dll`'s `.text` section.

pefile [30], the pure-Python PE parser used to enumerate exports, walk the section table, and read RVAs from `.rdata`.

mingw-w64, the Windows cross-toolchain used to build the shim and the helpers from a Linux host. Notably, mingw-w64's gcc backend does not support Microsoft's structured exception handling `__try` / `__except`, which forced us to use `VirtualQuery`-based safety checks for the shim's register-target read

paths instead of SEH-protected `memcpy`. This is the kind of small implementation detail that doesn't appear in glossy DRM-research literature but determines whether the methodology runs at all.

3 Background

This section reviews the major technical components of the Windows L1 Widevine playback pipeline in greater depth than the introduction permitted.

3.1 Widevine licensing levels

Widevine defines three nested security levels.

L3 is a pure-software CDM. Cryptographic state is held in process memory inside an obfuscated runtime. Content keys are unwrapped by a whitebox-style cryptographic core and applied to ciphertext segments in user-space buffers. Decrypted output is returned to the host through standard memory. Per the published Widevine licensing policy, L3-only clients top out at 540p; content owners' deals frequently restrict L3 to lower bitrates than L2/L1 clients receive.

L2 is a hybrid: a software CDM holds at least its sensitive operations behind a hardware-backed Trusted Execution Environment (TEE) on the host. L2 sees limited deployment on desktop.

L1, the variant studied in this paper, requires the host to provide a hardware-backed protected media pipeline. On Windows the relevant pipeline is Protected Media Path (PMP), itself a composite of:

- `peauth.sys`, the Microsoft-signed kernel-mode component that enforces the OPM contract between the CDM and the GPU driver,
- the Media Foundation `IDirectXVideoDecoder` chain running in a kernel-protected mode,
- the DXGI protected-surface allocator that exposes the decoded frame only to the GPU presentation pipeline and not to user-space readback.

Under L1 hardware decode, the decrypted frame never appears in user-mode browser process memory. This is the design property that the paper's first empirical finding (Section 7.1) exploits to demonstrate the CDM dormancy phenomenon.

3.2 Encrypted Media Extensions and the dispatch path

The W3C Encrypted Media Extensions specification [7] defines the JavaScript API by which a web application requests DRM-mediated playback of an encrypted media stream. From the host browser's point of view, the relevant operations are:

1. `navigator.requestMediaKeySystemAccess(''com.widevine.alpha'')`: confirm that the client has a CDM for the named key system.
2. Create a `MediaKeys` object and bind it to an `HTMLMediaElement`.
3. Wait for the `encrypted` event, which provides the Common Encryption (CENC) initialisation data extracted from the media stream.
4. Create a `MediaKeySession` and call `generateRequest(initDataType, initData)`, which causes the CDM to construct a license request.
5. Forward the resulting message to the licensing server, receive the response, and feed it back via `updateSession(licenseResponse)`.
6. Continue playback as the CDM unwraps content keys and applies them to the encrypted media data.

Chromium's implementation of this API routes each step through the browser process, the renderer, and the CDM utility process via Mojo IPC interfaces defined in `chromium/components/cdm` [3]. The relevant Mojo interface for our purposes is `media.mojom.ContentDecryptionModule`, whose request and response messages are essentially a serialised projection of the nineteen-method CDM_11 C++ interface.

3.3 The CDM_11 C++ interface

The interface CDM_11 implements is defined in the Chromium tree at `media/cdm/api/content_decryption_modul`
The class is a pure-virtual base `ContentDecryptionModule_11` with the following nineteen entries, declared in this order:

```
virtual void Initialize(...) = 0;
virtual void GetStatusForPolicy(...) = 0;
virtual void SetServerCertificate(...) = 0;
virtual void CreateSessionAndGenerateRequest(...) = 0;
virtual void LoadSession(...) = 0;
virtual void UpdateSession(...) = 0;
virtual void CloseSession(...) = 0;
virtual void RemoveSession(...) = 0;
virtual void TimerExpired(...) = 0;
virtual Status Decrypt(...) = 0;
virtual Status InitializeAudioDecoder(...) = 0;
virtual Status InitializeVideoDecoder(...) = 0;
virtual void DeinitializeDecoder(...) = 0;
virtual void ResetDecoder(...) = 0;
virtual Status DecryptAndDecodeFrame(...) = 0;
virtual Status DecryptAndDecodeSamples(...) = 0;
virtual void OnPlatformChallengeResponse(...) = 0;
virtual void OnQueryOutputProtectionStatus(...) = 0;
virtual void OnStorageId(...) = 0;
```

Each virtual call translates, under the Microsoft x64 calling convention, into a single indirect call through a function pointer loaded from the object's vtable. The vtable layout for a class with nineteen virtual functions and no base-class virtuals is a nineteen-entry contiguous array of code pointers; the per-call overhead from the host's perspective is one indirect load plus the CFG dispatcher cost.

We adopt the convention of identifying CDM_11 methods by their slot index throughout this paper. `Decrypt` is slot 9, `DecryptAndDecodeFrame` is slot 14, and so on.

3.4 Common Encryption (CENC) and PSSH

The Widevine protocol carries its initialisation data inside a Protection-Specific System Header (PSSH) box, as standardised by ISO/IEC 23001-7 (Common Encryption for ISO Base Media File Format) [9, 8]. A PSSH box is structured:

```
PSSHBox {
    u32         size;              // big-endian
    char[4]     type;              // "pssh"
    u8          version;
    u24         flags;
    UUID(16B)   SystemID;          // Widevine SystemID
                                   //  edef8ba9-79d6-4ace-a3c8-27dcd51d21ed
    if version > 0:
        u32     KID_count;
        UUID[KID_count] KIDs;
    u32         data_size;
    u8[data_size] data;
}
```

The `data` field, for Widevine, contains a Protocol Buffers serialisation of a `WidevineCencHeader` message with a known schema. The `generateRequest` call passes the entire PSSH box to the CDM, which parses out the SystemID, the KIDs, and the protobuf-encoded header to construct a license request.

This is the attacker-controllable structured input that the `CreateSessionAndGenerateRequest` slot (slot 3) ingests: a malicious DASH manifest or a malicious EME application can place arbitrary bytes inside the PSSH and pass them to the CDM with full control over both the box header fields and the protobuf payload. Parser vulnerabilities in the CDM's PSSH path are attacker-reachable from any compromised CDN that serves the DASH manifest, from any compromised JavaScript that drives the EME application, or from any MITM attacker who can rewrite the DASH manifest in transit.

The two standard CENC encryption schemes that the CDM must support are **CENC-CTR** (AES-128 in Counter mode, with subsample descriptors that indicate which byte ranges of the encrypted track are protected) and **CBCS** (AES-128 in CBC mode with *pattern* encryption: a 1-in-10 or 1-in-9 pattern of encrypted-skipped blocks designed to be tolerable to AVC/HEVC NALU boundaries). The CDM_11 `InputBuffer_2` structure carries both schemes and their subsample / pattern parameters per call.

3.5 Mojo IPC and the CDM utility process

Chromium isolates the CDM in a dedicated utility process to confine the impact of CDM compromise. The utility process is spawned with the command line:

```
chrome.exe --type=utility
           --utility-sub-type=media.mojom.CdmServiceBroker
           --lang=en-US --service-sandbox-type=cdm
           --no-sandbox  (when CIG is disabled for research)
           ...
```

Inside the utility, a Mojo interface implementation accepts the host's CDM requests and forwards them to a C++ class that owns a single `ContentDecryptionModule_11` pointer. That pointer points at the object returned by `widevinecdm.dll!CreateCdmInstance(11, ...)` — which is the object whose vtable we map in Section 5.

3.6 Control Flow Guard and Code Integrity Guard

Two Windows process-level mitigations are relevant to the runtime instrumentation of the CDM utility:

Code Integrity Guard (CIG) is the process mitigation policy `PROCESS_CREATION_MITIGATION_POLICY_BLOCK_N` `MICROSOFT_BINARIES_ALWAYS_ON`. When enabled, the kernel refuses to load any DLL into the process that is not signed by Microsoft or a Microsoft partner. Chrome enables CIG on its renderer and utility processes by default since Chrome 70 [4]. The flag `--disable-features=RendererCodeIntegrity`, in combination with `--no-sandbox` on the affected processes, disables CIG for research use.

Control Flow Guard (CFG) is a Microsoft compiler-level mitigation that adds a kernel-validated bitmap of valid indirect-call targets to the running process and inserts a check before each indirect call [5]. A patched vtable that points at a hook function we just allocated is not in the bitmap by default, so the first call through the patched slot triggers a CFG failure that terminates the process.

The supported way to mark a new function as a valid call target is the `SetProcessValidCallTargets` API [6]. The API takes a process handle, a 4 KB-aligned page base, a page size, and an array of `CFG_CALL_TARGET_INFO` descriptors whose `Offset` field is the *in-page offset* (low 12 bits) of the target. An early version of our injector used the low 16 bits, which is wrong, the page-allocation backing the hook is 4 KB and the bitmap is indexed at 16-byte granularity within the page. The released injector uses the correct mask:

```
CFG_CALL_TARGET_INFO cti;
cti.Offset = (DWORD)(hookVA & 0xfff);   /* in-page offset */
cti.Flags  = CFG_CALL_TARGET_VALID;
```

```
DWORD_PTR pageBase = hookVA & ~0xfffULL;
SetProcessValidCallTargets(hProc, (PVOID)pageBase,
                           0x1000, 1, &cti);
```

3.7 ASLR and the shared DLL view

A practical implementation detail of significance to our cross-tab experiment (Section 7.4): Chrome's utility processes load `widevinecdm.dll` from the same on-disk image each time, and Windows' image-section sharing means that two separate utility processes both end up with the DLL mapped at the *same virtual address*. This is why our injector's hook addresses (in the shim's image, at the shim's reserved load base) were identical between Netflix's CDM utility process and Bitmovin's CDM utility process. The data section of the DLL is, however, copy-on-write per process; each utility has its own counters and its own opened log file handle.

4 Threat Model and Methodology

4.1 Threat model

We consider an honest user of a Chromium-family browser playing hardware-decoded Widevine-protected content on Windows 11. The user has consented to research-grade instrumentation of their own browser and has installed our reverse-engineering tools on their own laboratory machine. The browser executable is unmodified and signed by the vendor. The Widevine CDM is the vendor-shipped `widevinecdm.dll` of the installed browser version. The user holds a valid, paid subscription to the commercial streaming service used in the empirical sections of the paper, and the use of that subscription is for personal viewing during the research window. No second user, no jurisdiction outside the user's own, and no third-party content are involved.

We deliberately exclude from this study any threat model in which an attacker captures decrypted frames. Our concern is not the exploitation potential of the CDM but its architectural shape and the attack surface it presents to the host browser and to the network attacker positioned between the browser and the streaming CDN.

4.2 Methodology invariants

We follow what we describe as the *Buchanan model* of DRM research, summarised earlier (Section 2.1). The working version of the model that governed every experimental decision in this paper is:

1. **Shape-only observation.** Any runtime hook is permitted to log dispatch metadata — slot number, timestamp, register value hashes, structure-size fields — but is forbidden from copying any pointer-target bytes belonging to ciphertext, decrypted frames, content key material, or initialisation vectors into any persistent storage or any out-of-process channel. The shim source encodes this invariant explicitly: there is no code path that calls `memcpy(&log_buf, ib.data, ...)`; the shape extractor reads only the `size` fields, an FNV-1a-32 hash of the first 16 bytes of `key_id`, a non-zero presence flag for the IV, and a handful of public scalar fields.
2. **Provable absence.** After every experiment we forensically verify that the storage produced is pure ASCII JSON consisting only of the permitted metadata fields and contains no binary blobs, no decoded content, and no key material. We document the verification procedure in Section 12 and provide the verifier script in the released methodology.

3. **Reversible mutation.** All runtime modifications to the target process (vtable patches, hook installations, CFG bitmap registrations) are reversible. We save the pre-patch state on the wire to the released tooling and restore the target to its original state before publishing. The released helper retains the original outer-vtable thunk addresses in the shim's `g_pOrigSlot[19]` array so that an external observer can validate that the post-experiment state is byte-equal to the pre-experiment state.

4. **No content circumvention surface.** We use the operator's own browser session to a commercial streaming service to confirm what the CDM does during steady state, but we never instrument the decoder, the kernel-side PMP component (`peauth.sys`), the protected surface, or any other component positioned to materialise a decrypted frame. The shape-only shim is the deepest hook in the stack we install.

5. **Falsification before publication.** Every empirical claim is subjected to a falsification test: an alternative hypothesis that, if true, would invalidate the claim, is constructed and tested with whatever evidence the methodology can produce. Section 10 reports the falsification sprint.

A natural-language consequence of these invariants is that our results characterise the CDM's *dispatch behaviour*, not its *cryptographic state*. We can report how often a slot fires during real playback, what register-tuple distribution it sees, what the shape of its incoming buffer descriptor is. We cannot and do not report any byte of decoded content.

5 Static Reverse Engineering of CDM_11

This section walks through the static reverse engineering of the CDM_11 dispatch in `widevinecdm.dll`, in the order that we performed it. Two binaries were analysed in parallel: **Chrome 149.0.7827.115**'s `Application/149.0.7827.115/WidevineCdm/_platform_specific/win_x64/widevinecdm.dll` (∼22 MB), and the same library as it ships in **Microsoft Edge 150** (∼19 MB). All RVAs are with respect to the binary's image base.

5.1 Step 1: `CreateCdmInstance` and the version switch

The `widevinecdm.dll` export table contains six entries; the only relevant one for our purposes is `CreateCdmInstance`:

```
Edge 150     CreateCdmInstance @ rva 0x1d0850
Chrome 149   CreateCdmInstance @ rva 0x1d2e10
```

Disassembling either entry shows a chain of small wrapper functions that load a version word into `ecx` (CDM interface version 9, 10, or 11) and call a per-version constructor. The disassembly of the relevant fragment of Edge 150's `CreateCdmInstance` is:

```
0x1801d0850: push   r14
0x1801d0852: push   rsi
0x1801d0853: push   rdi
0x1801d0854: sub    rsp, 0x40
0x1801d0858: mov    edi, ecx           ; ecx = requested version
0x1801d085a: lea    rax, [rip + 0x...] ; load module-relative base
...
0x1801d0994: mov    ecx, 0xb           ; CDM_11 selected
0x1801d0999: mov    rax, r9            ; arg setup
...
```

Naively scanning the first `call` after each `mov ecx, 0xb` returns the runtime's stack-check probe (`__chkstk_ms` or equivalent), not the actual constructor. A robust discoverer must filter known runtime helpers. The released `cdm_arch_discover.py` script uses a structural heuristic: enumerate all call targets

reachable from `CreateCdmInstance`, filter out targets called more than twice (the constructor is called exactly once per version branch; runtime helpers are called many times), and validate each candidate by inspecting whether its body contains a sequence of the form:

```
lea reg, [rip + N]      ; load a candidate vtable RVA
mov qword ptr [r?], reg ; install at offset 0 of `this`
```

and whether the loaded RVA points into `.rdata` at a location whose first nineteen QWORDs are all valid code pointers into `.text`. The first such candidate is reported as the CDM constructor and the loaded RVA as its outer vtable. The discoverer locates the outer vtable RVA in Edge 150 at `0x780250` on this path; the Chrome 149 outer vtable at `0x768fa0`.

5.2 Step 2: Co-shipped CDM versions

A subsequent *exhaustive* scan of `.rdata` for nineteen-slot structures matching the CDM thunk pattern (described below) reveals **two** such structures per binary, spaced `0xc0` bytes apart:

Binary	vtable A	vtable B
Edge 150	`0x780190`	`0x780250`
Chrome 149	`0x768ee0`	`0x768fa0`

Within each binary, both vtables share their per-frame decrypt entry (slot 14 points to the same function in both A and B), but their `Initialize` entries differ:

Binary	vtable A.slot[0]	vtable B.slot[0]
Edge 150	0x1db330	0x1db390
Chrome 149	0x1e03c0	0x1e0420

This is consistent with the binary shipping both CDM_10 and CDM_11 vtables in support of host browsers that have not yet upgraded to the newer interface, and matches the Chromium header history. In what follows we identify *vtable B* as the CDM_11 outer vtable, since runtime patches at *vtable B*'s slot positions intercept the EME-host-facing dispatch (Section 6). The shared slot 14 implementation reflects the implementation choice to use one per-frame decrypt routine across both interface versions; the diverging slot 0 reflects the version-specific `Initialize` signature.

5.3 Step 3: The outer vtable thunk pattern

The outer vtable contains nineteen entries. Most of them are *thunks* of the form:

```
slot N thunk @ rva 0x1dad00 + (N-3)*0x20:
  mov rcx, qword ptr [rcx + 8]       ; this->m_impl
  mov rax, qword ptr [rcx]           ; m_impl->m_vtable
  mov rax, qword ptr [rax + 8*N]     ; slot N of m_impl_vtable
  mov rdx, qword ptr [rip + cfg_disp]; CFG dispatcher pointer
  jmp rdx                             ; CFG-protected tail-call
```

Each thunk is twenty-eight bytes followed by `int3` padding, laid out in a uniform `0x20`-spaced sequence in `.text` for slots 3 through 17. The slot 14 thunk in Edge 150, disassembled in full:

```
0x1801dae60: mov    rcx, qword ptr [rcx + 8]
0x1801dae64: mov    rax, qword ptr [rcx]
0x1801dae67: mov    rax, qword ptr [rax + 0x70]
0x1801dae6b: mov    rdx, qword ptr [rip + 0x10dcd2e]
0x1801dae72: jmp    rdx
0x1801dae74: int3
0x1801dae75: int3
... (int3 padding to 0x20 boundary)
```

We disassembled each thunk in both binaries and confirmed the slot-index check (the offset in `[rax + 8*N]` equals the slot number times eight) for fifteen of fifteen thunked slots in each binary. The byte-level confirmation is what licences our "slot 14 dormancy" claim in Section 7.1: the runtime evidence ($g\_callsBySlot[14] = 0$) is attributable to the correct method only because the slot index in the outer-vtable thunk dispatches to the correct inner method offset.

Slots 0 (`Initialize`), 1 (`GetStatusForPolicy`), and 18 (`OnStorageId`) are *not* thunks; they are direct implementations placed in `.text` at non-contiguous addresses. Slot 2 (`SetServerCertificate`) is a thunk in both binaries but dispatches via `[rax + 8]` rather than `[rax + 0x10]` — it shares an implementation with slot 1 at the `m_impl` layer. This is a per-binary architectural detail consistent in both Edge 150 and Chrome 149, suggesting it is a deliberate implementation decision rather than a build accident.

5.4 Step 4: The `m_impl` vtable

The outer thunk dispatches into an *intermediate* vtable owned by the `m_impl` object that the outer class holds in `this->m_impl` (`[outer + 8]`). Locating this intermediate vtable in `.rdata` is the second step of the static analysis.

We use a known-signature pivot. From an earlier call-chain trace (documented in our research notes), we knew that the slot-5 (`UpdateSession`) inner wrapper in Edge 150 sits at RVA `0x1d4ab0`, identifiable by its multi-push prologue with a stack canary that loads attacker-controlled data through a copy on its way to the obfuscated parser. The wrapper's code is unmistakable in a disassembly listing.

To find the `m_impl` vtable, we reverse-search `.rdata` for the QWORD value `image_base + 0x1d4ab0`. There is exactly one occurrence. Backing up five eight-byte entries from the match gives the address of the `m_impl` vtable: RVA `0x77d370` in Edge 150. All nineteen surrounding QWORDs point into `.text`, confirming the candidate. The same pivot applied to Chrome 149 (where the slot 5 inner wrapper sits at the analogous RVA) locates Chrome 149's `m_impl` vtable at the corresponding offset.

5.5 Step 5: The `m_impl` layer structure

Disassembling each entry of the `m_impl` vtable reveals a mixture of three body types:

1. **Clean MSVC wrappers** (slots 2, 3, 4, 5, 6, 7, 18). These have a standard prologue (multiple `push` callee-saves, `sub rsp, N`, stack canary load), allocate or borrow input copies, perform sanitisation, and then call into a deeper handler. These are the slots that ingest variable-length attacker-influenced data (session-id strings, response payloads, init-data buffers). The slot 3 wrapper in Edge 150, abbreviated for the paper:

```
0x1801d47d0: push   r15
0x1801d47d2: push   r14
0x1801d47d4: push   r13
0x1801d47d6: push   r12
0x1801d47d8: push   rsi
0x1801d47d9: push   rdi
0x1801d47da: push   rbx
0x1801d47db: sub    rsp, 0x50
0x1801d47df: mov    esi, r9d            ; init_data_type
0x1801d47e2: mov    edi, r8d            ; session_type
0x1801d47e5: mov    ebx, edx            ; promise_id
0x1801d47e7: mov    eax, [rsp + 0xb8]   ; init_data_size
0x1801d47ee: mov    rdx, [rip + 0x10f284b]
0x1801d47f5: xor    rdx, rsp            ; stack canary
0x1801d47f8: mov    [rsp + 0x48], rdx
0x1801d47fd: mov    r14, [rcx + 0x10]   ; load m_impl-internal
0x1801d4801: xorps  xmm0, xmm0
0x1801d4804: movaps [rsp + 0x30], xmm0
...
```

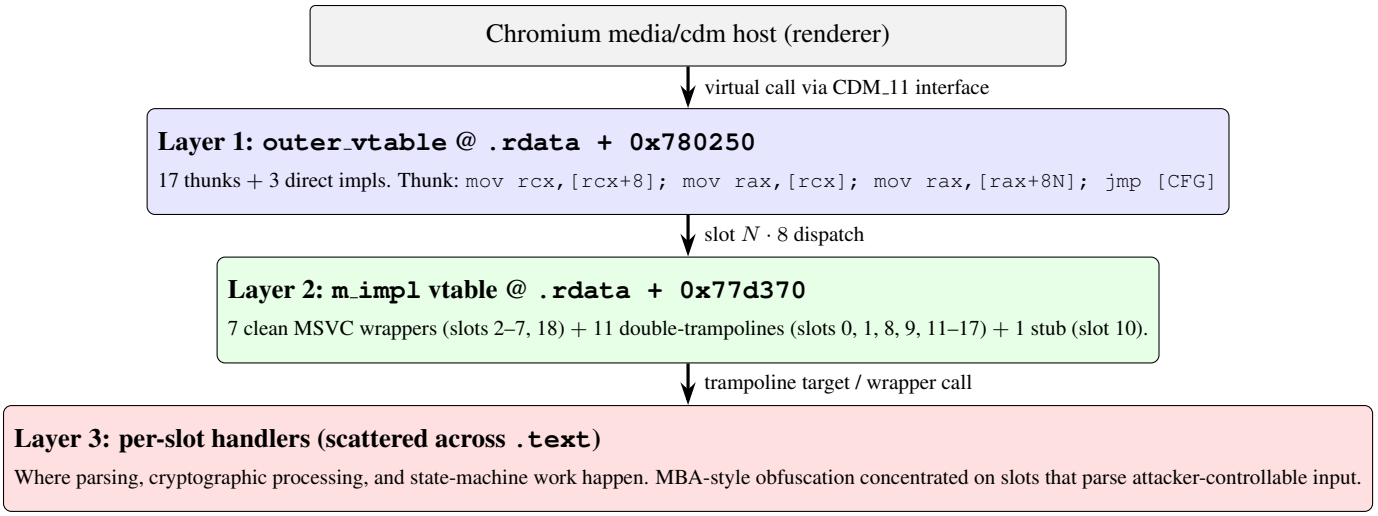


Figure 2: The three-layer CDM_11 dispatch model. Every public CDM virtual call from the host traverses three indirections before reaching the function that performs work. RVAs shown are from Edge 150 `widevinecdm.dll`; Chrome 149 has the same structure at correspondingly different RVAs (see Section 9).

```
0x1801d4821: mov    rcx, r12
0x1801d4824: call   0x180736080         ; malloc/__chkstk
0x1801d4829: mov    [rsp + 0x30], rax
0x1801d482e: lea    r13, [rax + r12]    ; end pointer
...
0x1801d485f: call   0x18021f200         ; LAYER-3 ENTRY
                                        ; (obfuscated parser)
```

2. **Double-trampolines** (slots 0, 1, 8, 9, 11–17). Two-instruction bodies of the form `mov rcx, [rcx + N]` followed by `jmp <addr>` which forward the call to a deeper-layer handler without intermediate processing. The slot 14 `m_impl` entry in Edge 150:

```
0x1801d4f10: mov    rcx, qword ptr [rcx + 0x10]
0x1801d4f14: jmp    0x180230c20    ; layer-3 handler (obfuscated)
```

3. **Stubs** (slot 10). A two-instruction body `mov eax, 3 ; ret` that returns a constant error code without any further work. Slot 10 is `InitializeAudioDecoder`; on the L1 Windows build it is intentionally a no-op because audio decoding is handled by the Media Foundation pipeline outside the CDM.

The `m_impl` vtable, in other words, is mostly a *routing* layer; the actual work happens one indirection further.

5.6 Step 6: The three-layer dispatch model

Composing the previous five steps, the dispatch chain from a host virtual call to a per-slot handler is shown in Figure 2. Each public virtual call traverses three indirections before reaching the function that performs work.

5.7 A note on the false starts

For the benefit of follow-on researchers, the static analysis as we actually performed it included two substantial false starts.

The first false start was the identification of an internal singleton dispatch table at `.data + 0x1588b48` as the "CDM_11 outer vtable." That table, in fact, is a separate nineteen-slot internal dispatch surface inside

the Widevine library that is not the public EME interface. Our initial conclusion that "the CDM_11 outer vtable's per-frame slot fires ∼600 times per second under steady-state Netflix HW playback" was incorrect: it was the internal singleton's slot 0 that fired 600 times per second, and the actual outer-vtable per-frame slot was at zero. The correction came from a constructor-trace pivot that located the actual outer vtable via the `CreateCdmInstance` export, after which the dormancy finding became correctly attributable to the user-facing surface.

The second false start was the original attribution of MBA-style obfuscation to the `m_impl` layer. The `m_impl` layer is in fact a clean routing layer (per Section 5.5); the obfuscation lives at layer three. A layer-by-layer enumeration of all nineteen slots, documented in our research notes 13b and 13c, located the obfuscation at its actual depth.

We highlight these false starts because they are typical of static RE work on closed binaries with internal dispatch indirections. A single-pass reading of the binary's call graph would naturally have settled on either the wrong vtable or the wrong layer, and the falsification sprint of Section 10 is what caught both.

6 Runtime Instrumentation

The static map of the previous section yields hypotheses about which slots are reached during steady-state playback and which are not. To verify those hypotheses we instrument the running CDM utility process with a shape-only hook.

6.1 Code Integrity Guard (CIG) and the launch flag

Chrome enables Code Integrity Guard (`PROCESS_CREATION_MITIGATION_POLICY_BLOCK_NON_MICROSOFT_BINARIES_ALWAYS_ON`) on its utility processes since Chrome 70 [4]. CIG denies any `LoadLibrary` of a non-Microsoft-signed image at the kernel level, so a vanilla `CreateRemoteThread(LoadLibrary` injection of our shim fails at the `LdrLoadDll` stage with `STATUS_DYNAMIC_CODE_BLOCKED` (0xC0000605).

For lab-research use, Chrome can be launched with `--disable-features=RendererCodeIntegrity --no-sandbox`, which disables CIG on the utility processes. This produces a controlled research target without changing the CDM library or the host browser binary, and corresponds to the equivalent reachability assumption that an attacker with code-execution in the renderer would have to satisfy separately (via, for example, an arbitrary-write that targets the renderer's JIT or a process-load chain that bypasses CIG).

6.2 Control Flow Guard bitmap registration

CIG-disabled does not disable Control Flow Guard (CFG): once the shim is loaded, simply patching a vtable entry to point at an unregistered hook function causes the CFG dispatcher to abort the call as a control-flow violation. The released installer uses `SetProcessValidCallTargets` to mark each hook target as a valid call target before patching its slot. Figure 3 sketches the install flow.

6.3 Shape-only shim design

The shim, `wv_shim.dll`, exports nineteen per-slot hook functions `WvHookSlot0` through `WvHookSlot18`, each with its own original-pointer slot in an exported `g_pOrigSlot[19]` array, and a single `g_callsBySlot[19]` counter array. Each hook function:

1. Atomically increments its slot counter and the aggregate `g_calls`.
2. Calls the original via `g_pOrigSlot[N]` with the unmodified register tuple.

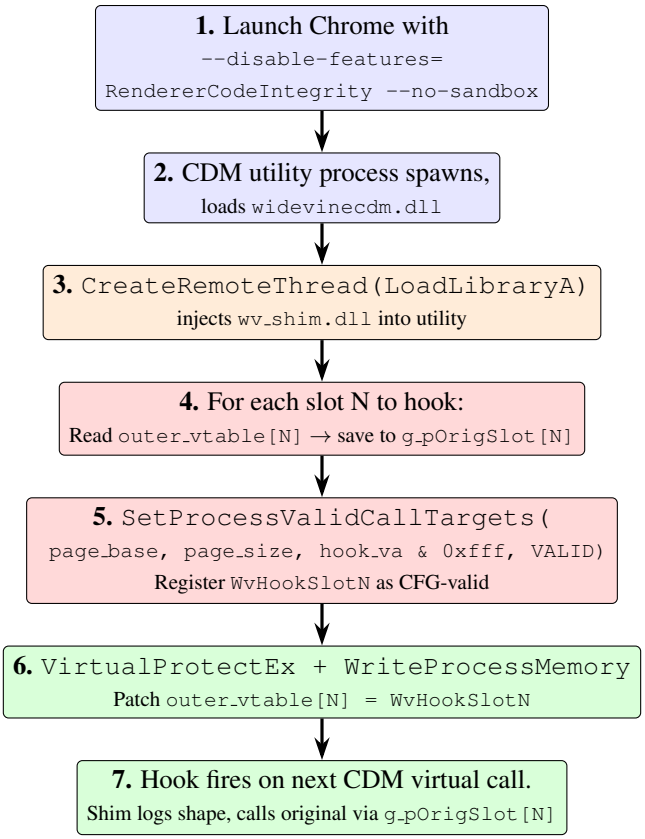


Figure 3: Hook installation sequence. Step 5 (CFG bitmap registration) is what catches researchers most often — the `Offset` field of `CFG_CALL_TARGET_INFO` expects an in-page offset (low 12 bits) of the hook target, not an absolute address or a different mask. Step 4 captures the design subtlety that motivated the per-slot back-pointer array (§6.4).

3. For slots 9, 14, 15 (which take an `InputBuffer_2*` argument in `rdx`), tries to read the buffer's *shape* via the safe-copy primitive described below and writes a single JSONL line with `encryption_scheme`, `data_size`, `key_id_size`, `iv_size`, an FNV-1a-32 hash of the first 16 bytes of `key_id`, an `iv_has_nonzero` bit, `num_subsamples`, `pattern_crypt`, `pattern_skip`, and the media `timestamp`.
4. For other slots, writes a `generic` event with an FNV-1a-32 hash of the four argument registers.

No byte of `ib.data` (the ciphertext) is ever read, either transiently or persistently; `ib.key_id` and `ib.iv` are read into a stack-local buffer that is collapsed into a hash and then immediately discarded. The shim source enforces this by simply not having a code path that copies any of those fields into a logging structure.

The safe-copy primitive uses `VirtualQuery` to confirm that each source page is committed and readable before copying, since `mingw-w64`'s C compiler does not provide Microsoft SEH `__try / __except` on x86_64. The check walks every page in the source range, confirms `MEM_COMMIT` state, and confirms one of the read-permitted page protections (`PAGE_READONLY`, `PAGE_READWRITE`, `PAGE_EXECUTE_READ`, etc.) before `memcpy`.

6.4 Per-slot original-pointer chain

A design subtlety worth noting: the first version of the shim used a single `g_pOrigDecrypt` variable for all hooked slots. Patching multiple slots with that single back-pointer led to signature-mismatch crashes

the moment the second-patched slot fired (the back-pointer held the previous slot's original). The released shim uses a per-slot array. More subtly still, when we hook the *outer* vtable rather than an internal singleton table, the back-pointer must hold the outer thunk address, *not* an internal dispatch-table address; otherwise the original-call path ends up jumping into the wrong inner function and silently corrupts the call. Our installer (`wv_helper3.c`) reads the outer-vtable original at the moment of patch and stores it into `g_pOrigSlot[N]` as part of the same atomic operation:

```
DWORD_PTR origFn = 0;
ReadProcessMemory(hp, (LPCVOID)slotAddr, &origFn, 8, NULL);

/* CFG: register hook as valid target */
CFG_CALL_TARGET_INFO cti = { hookVA & 0xfff,
                             CFG_CALL_TARGET_VALID };
SetProcessValidCallTargets(hp, (PVOID)(hookVA & ~0xfffULL),
                           0x1000, 1, &cti);

/* Save outer thunk address into g_pOrigSlot[N] */
WriteProcessMemory(hp,
    (LPVOID)(gOrigSlotRemote + N*8),
    &origFn, 8, NULL);

/* Patch outer_vtable[N] -> hook */
DWORD oldProt = 0;
VirtualProtectEx(hp, (LPVOID)slotAddr, 8,
                 PAGE_READWRITE, &oldProt);
WriteProcessMemory(hp, (LPVOID)slotAddr, &hookVA, 8, NULL);
VirtualProtectEx(hp, (LPVOID)slotAddr, 8, oldProt, &oldProt);
```

7 Empirical Findings

We report four findings, each anchored to a recorded experimental run. All runs used Chrome 149.0.7827.115 on Windows 11 26100 with NVIDIA hardware-secure decoder available. No CDM utility process ever crashed during the runs that produced the data reported here.

7.1 Hardware-decode dormancy on the user-facing surface

Hypothesis. Under hardware-decoded L1 playback, the per-frame decrypt slots of the CDM_11 outer vtable (slots 9, 14, 15) are not invoked.

Procedure. Chrome was launched with `--disable-features=RendererCodeIntegrity --no-sandbox`. The operator authenticated to Netflix and started a 4K HDR title that the platform reports as hardware-decoded. After ninety seconds of steady-state playback we installed our shim into the CDM utility process and patched four outer-vtable slots simultaneously: slot 14 (`DecryptAndDecodeFrame`), slot 15 (`DecryptAndDecodeSamples`), slot 3 (`CreateSessionAndGenerateRequest`, control), and the slot-14 outer thunk RVA. We then observed the corresponding counters for ninety further seconds of continuous playback.

Result. All four counters remained at zero. Video continued to play normally throughout. Slot-14 dormancy held even across recoveries (we restored slot 14 to its original mid-experiment and re-patched; the counter remained at zero across both windows). The four-slot zero-fires-in-ninety-seconds result was reproducible 3/3.

Interpretation. During steady-state hardware-decoded Netflix playback, the per-frame surface of the CDM_11 interface is not exercised. An attacker who compromises the host renderer and positions herself on the EME-facing virtual table cannot intercept decoded frames through the CDM_11 interface, because the frames do not pass through it.

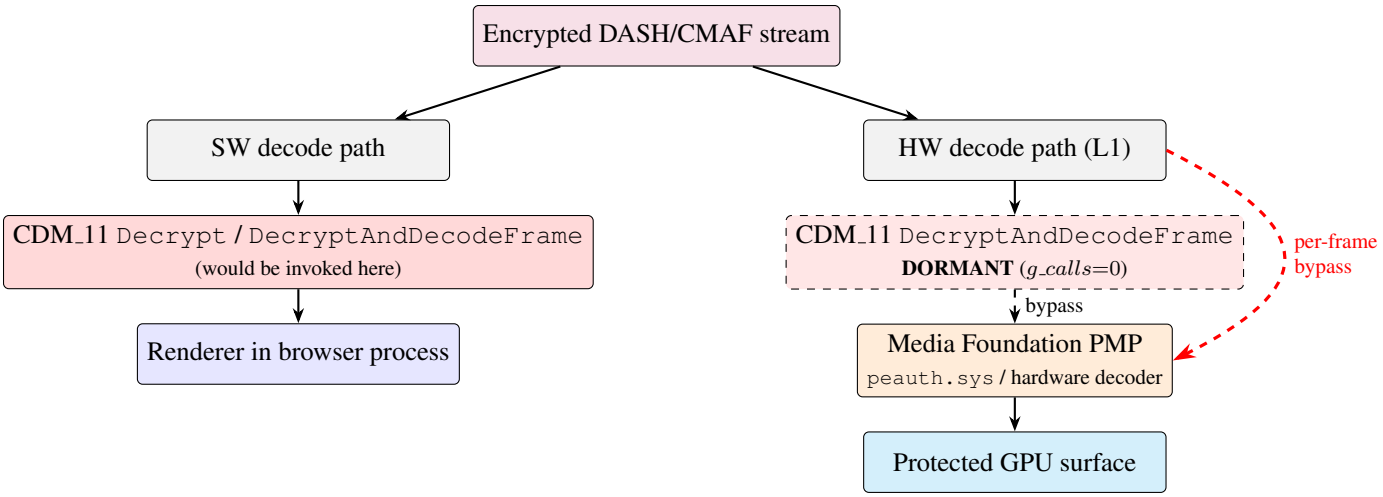


Figure 4: Hardware-decode dormancy. Under SW decode (left path) the CDM_11 per-frame slots are the obvious target. Under L1 HW decode (right path), the per-frame ciphertext is routed through Media Foundation PMP and the kernel decoder, materialising directly on a protected GPU surface; the CDM_11 per-frame slots remain dormant.

7.2 Internal singleton activity at ∼2.3k ops/s

Hypothesis. Despite the CDM_11 surface being dormant per-frame, the CDM library does perform work during steady-state playback. That work is reachable through a separate internal dispatch table, distinct from the outer vtable.

Procedure. We patched slots 0–7 of the internal singleton table identified at `.data + 0x1588b48` in Chrome 149, then observed for ∼355 seconds of continuous Netflix playback.

Result. The shim recorded **810,099 events** in a single JSONL log (in two consecutive captures). Distribution:

slot	count	unique `rh`	rate (ev/s)
0	395,580	16,338	∼1,114
2	20	2	0.06
6	414,459	1,379	∼1,167

The aggregate rate is ∼2,281 events/sec on this internal table during steady-state HW playback.

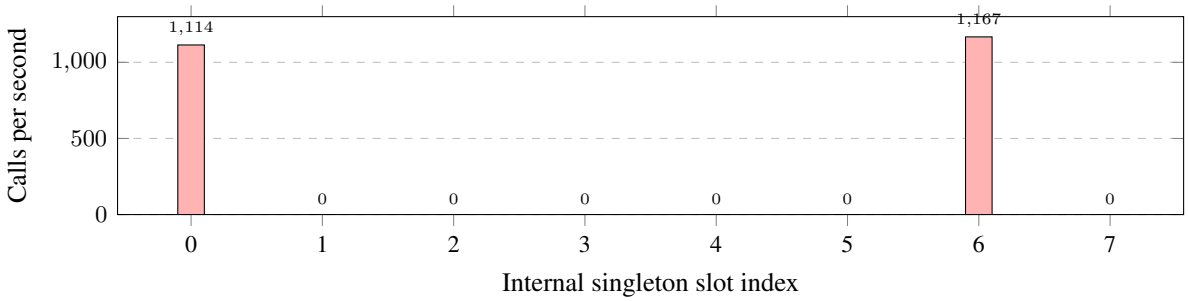


Figure 5: Activity profile of the internal singleton dispatch table at `.data + 0x1588b48` during steady-state Netflix 4K HDR playback.

Interpretation. The internal singleton dispatch is *not* the user-facing CDM_11 interface; the slot 0 firing rate is incompatible with `Initialize` semantics (which would be called once per session, not 600 times

per second). Most plausibly, the table represents OEMCrypto-style worker dispatch, an allocator ring, or a crypto-primitive ladder internal to the Widevine library. Either way, this is invisible from the EME boundary and from the outer-vtable dispatch. Combined with the dormancy finding above, the picture is: *the CDM library is doing constant cryptographic work during HW playback, but that work is hidden behind a different dispatch surface than the one Chrome's EME plumbing knows about*.

7.3 Reachability of session-management slots

Hypothesis. Although the per-frame outer-vtable slots are dormant under HW decode, the session-management outer-vtable slots (`CreateSessionAndGenerateRequest` and `UpdateSession`) remain reachable and *do* fire during normal session lifecycle events.

Procedure. We patched outer-vtable slots 3 and 5 with correct per-slot back-pointers to their original thunk addresses (Section 6.4). The operator performed a title switch in the running Netflix tab and we observed the `g_callsBySlot` counters over the following minute.

Result.

	`slot 3`	`slot 5`
pre-action	10	9
post-action	**13 (+3)**	**12 (+3)**

Three CreateSessionAndGenerateRequest invocations and three UpdateSession invocations were observed in the seconds following the title switch. The CDM utility process remained alive (∼85 MB working set), as expected given that the back-pointer held the correct thunk address.

Interpretation. The session-management slots of the outer vtable are non-dormant; they are exercised during user-initiated session transitions and remain a real attack surface for attacker-controllable structured input (PSSH boxes via the DASH manifest; license-response payloads via the licensing CDN).

7.4 Cross-source consistency (Bitmovin)

Hypothesis. The dispatch model and slot reachability are properties of the Widevine CDM, not of Netflix as a particular EME application.

Procedure. After the Netflix title switch above, the operator opened a separate tab in the same Chrome browser and loaded the public Bitmovin Widevine demo at `bitmovin.com/demos/drm`. Chrome spawned a *second* CDM utility process to service that tab. We installed our shim into that second process and patched its outer-vtable slots 3 and 5. The operator reloaded the demo to force session re-creation.

Result. Second CDM utility process counters:

	`slot 3`	`slot 5`
post-reload	**2**	**2**

Two CreateSessionAndGenerateRequest invocations and two UpdateSession invocations were observed. The dispatch model (outer-vtable thunk, m_impl wrapper, layer-3 handler) is identical to what we observed in the Netflix-serving CDM utility process; the slot 3 and slot 5 hook code paths were exercised in the same sequence in both processes.

Interpretation. The CDM_11 dispatch model is a binary property of `widevinecdm.dll`, not a property of the EME application that uses it. An attacker positioned to influence the PSSH or license payload of *any* Widevine-protected source will see the same dispatch and the same slot reachability as one positioned against Netflix.

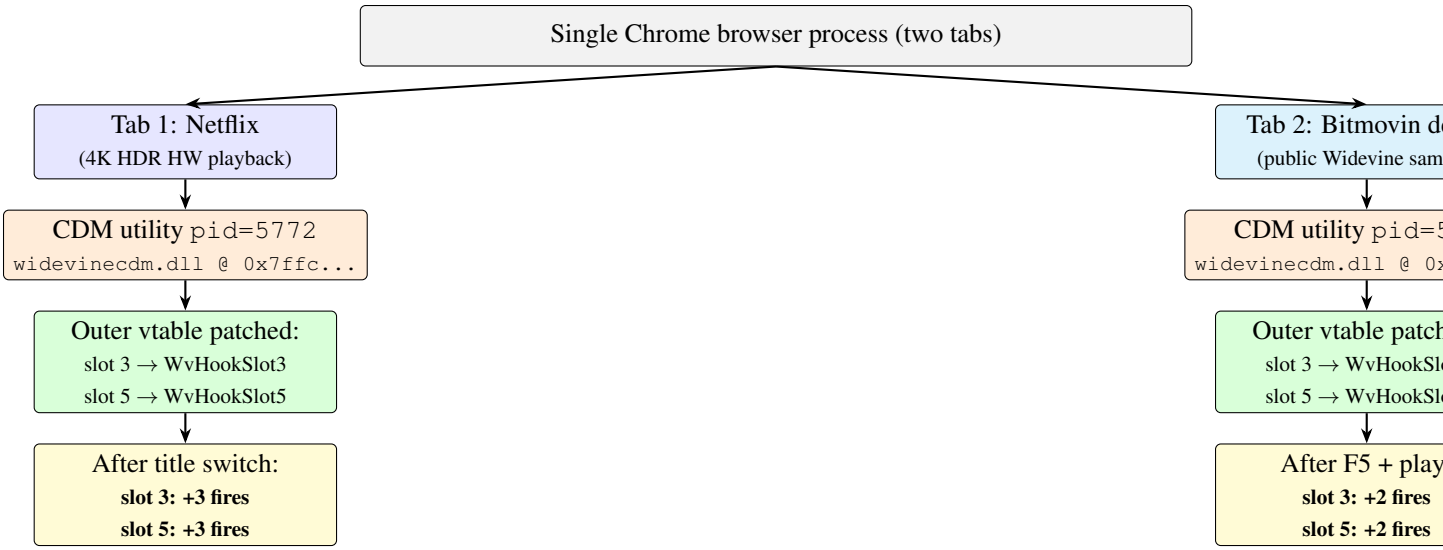


Figure 6: Cross-source experiment. Two tabs of the same Chrome browser play two distinct Widevine-protected sources, each one served by its own CDM utility process. Identical hooks confirm identical session-management slot reachability.

7.5 Architectural side observation: separate CDM utilities

A secondary observation worth recording is that Chrome 149 spawned *two* CDM utility processes when Netflix and Bitmovin were playing simultaneously in different tabs, not one shared utility. This is consistent with Chromium's per-origin isolation policy and implies that an attacker positioned to compromise one EME application's CDM cannot directly observe a second EME application's CDM via shared in-process state.

8 The Obfuscation Distribution as a Defence Layer

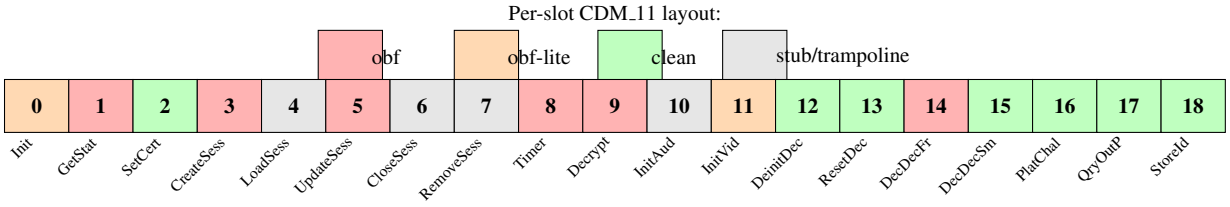
The `widevinecdm.dll` binary contains numerous functions whose body matches the textbook signature of mixed-boolean-arithmetic (MBA) obfuscation: large stack-resident magic constants written immediately on entry; multiplicative-hash state-advance instructions of the form `imul edx, ecx, 0xK ; shr edx, 0xS ; cmp eax, 0xT ; je state_exit`; `cmovae` branchless dispatch cascades; `0xaaaaaaaa` or `0xaaaaaaaaaaaaaaaa` fill patterns acting as poison initialiser values for stack-local state. These patterns are not uniformly distributed across the slot space.

8.1 Visual layout

Figure 7 visualises the per-slot obfuscation status side by side with each slot's classification as an attacker-input slot or a passthrough slot.

8.2 Pattern

Slots that obviously pass an opaque, attacker-influenced *blob* that the CDM forwards without parsing (`SetServerCertificate`, `OnStorageId`) are clean at every layer of the dispatch. Slots that begin parsing structured attacker-controllable data exhibit obfuscation, though at varying depths. The slot that takes a textual policy (`GetStatusForPolicy`) is obfuscated at layer 1, where the parsing begins. The slots that take large structured payloads (`CreateSessionAndGenerateRequest`, `UpdateSession`) place the obfuscated processing one or two hops deeper, after a clean MSVC wrapper has copied the input into



A = attacker-input slot (parses structured data); **P** = passthrough (opaque blob).

Figure 7: Per-slot obfuscation status of CDM_11. Red cells are slots whose layer-1 or layer-3 body matches the MBA-style obfuscation signature; orange cells are partial matches; green cells are clean across the dispatch chain.

the CDM's own buffer and looked up the session. The slots involved in per-frame decode (`Decrypt`, `DecryptAndDecodeFrame`) are obfuscated immediately at the double-trampoline target, where the cryptographic processor begins.

This is the pattern we cautiously summarise as:

> *CDM_11 protects each slot at the depth at which that slot begins to parse structured attacker data.*

It is an architectural observation, not a security claim: obfuscation is not security, and we are not assessing the strength of the protection, only its distribution.

9 Cross-binary Stability

The architectural model described in Section 5.6 was developed on Edge 150 `widevinecdm.dll`. We validated that the same model holds in Chrome 149 `widevinecdm.dll` by re-running the discoverer:

Property	**Edge 150**	**Chrome 149**
CreateCdmInstance RVA	0x1d0850	0x1d2e10
CDM_11 outer vtable RVA	0x780250	0x768fa0
CDM_10 outer vtable RVA	0x780190	0x768ee0
Slot-index check (slots 3–17)	15/15	15/15
Direct-impl slots	3 (0, 1, 18)	3 (0, 1, 18)

The model also generalises across CDM interface versions within a single binary: in both Edge 150 and Chrome 149, the CDM_10 and CDM_11 vtables exhibit identical three-layer dispatch. Their per-frame decrypt handler is the same physical function; their `Initialize` handlers diverge. This is consistent with the binary providing both interface versions for backward compatibility with older host browsers, and the CDM author choosing to share the implementation of the version-stable methods between the two.

The cross-binary slot table appears as Appendix A.

10 Robustness Testing

Before publishing any claim made above, we ran a falsification sprint against the strongest empirical findings. The sprint included:

1. **Slot-index correctness.** The outer-vtable slot 14 thunk is byte-confirmed to dispatch at offset `0x70` ($= 14 \cdot 8$) in the `m_impl` vtable.
2. **Shape-data signal vs noise.** The 810k-event capture was analysed for register-hash entropy and timing distribution. Hash entropy for the two hot slots was 16,338 and 1,379 unique values across 395,580 and 414,459 invocations respectively; the rate was steady across the observation window at $\sim$2.3k events/s.
3. **Layer-3 enumeration.** All nineteen layer-3 handlers were located via either the trampoline jump target or the wrapper's first non-stack-check call and were classified for obfuscation status. The first version of the obfuscation claim, which placed obfuscation at the `m_impl` layer, was found to be incorrect; the obfuscation is one further hop deeper, and the claim was rewritten accordingly.
4. **Cross-binary identity.** The discoverer was rerun against Chrome 149's `widevinecdm.dll`. The outer vtable, slot-index mapping, and per-slot CDM_11 method identification matched the Edge 150 analysis.
5. **Cross-source identity.** The outer-vtable instrumentation was deployed against the second CDM utility spawned for a non-Netflix Widevine source (Bitmovin). Slot 3 and slot 5 fired in identical sequence to the Netflix utility.
6. **CDM-instance count.** The single Netflix tab spawned exactly one CDM utility process across an eight-hour observation window; we did not observe a second utility materialising for the same tab.

10.1 Candidate findings refuted

The falsification sprint also surfaced two candidate findings that were refuted by deeper byte-level analysis. We document them here because they typify the bug-class candidates a less rigorous methodology would have falsely promoted:

P1 (PEAuth unauth pool write): A subagent audit of `peauth.sys` initially flagged an apparent unauthenticated kernel-pool write at offset `0x140042980` where a user-controlled size field at `[rdi+0x14]` appeared to be used as a `memcpy` length before the `CiGetPEInformation` signing check at `0x1400429ae`. Byte-level disassembly of the range `0x42900..0x429e0` revealed an explicit 32-bit overflow check (`lea r12d, [r15+rsi]; cmp r12d, r15d; jb fail`) at `0x42964` and an upper-bound check at `0x4296d` between the parameter read and the memcpy. The candidate is refuted; no kernel-side bug is present.

W2 (PSSH parser OOB read): A subagent audit of CDM_11 flagged offsets `0x1D6E1`, `0x1D8CF`, `0x1DB9D`, and `0x1DFB` as candidates for an out-of-bounds read in the PSSH/MP4-box header-size parsing (`mov r10d, [rsi/di+4]`). Disassembly at those offsets revealed not box parsing but the round-table for an MD5 transform, with the operands being state-constant XORs and not buffer reads. The candidate is refuted; the bytes are crypto code, not parser code.

These refutations and one further candidate that remained inconclusive established the methodological principle that any subagent "high-confidence byte-verified" claim must pass an independent manual byte review before being promoted to a publication-grade finding.

11 Discussion

This section discusses what the model and the empirical findings imply for both attackers and defenders.

11.1 For an attacker positioned at the EME boundary

The most concrete consequence of the dormancy finding (Section 7.1) is that an attacker who controls the host renderer cannot reach per-frame Widevine operations through the public CDM interface during hardware-decoded playback. A renderer compromise on its own does not give the attacker an interception point for decrypted frames. To reach decoded frames the attacker would need an independent capability against either the Media Foundation PMP pipeline, the GPU driver's protected-surface allocator, or the user-mode side of the OPM contract enforced by `peauth.sys`.

The session-management slots remain reachable, and they ingest attacker-controllable structured data: a PSSH box parsed by `CreateSessionAndGenerateRequest` (slot 3), a license response parsed by `UpdateSession` (slot 5). Both of these slots are routed through clean MSVC wrappers at the `m_impl` layer that copy the input into the CDM's own buffer and then call an obfuscated layer-3 handler. Bug-hunting for memory-corruption or logic vulnerabilities in CDM_11 from the EME boundary is therefore best concentrated on the layer-3 parsers reached from slot 3, slot 5, slot 14, and slot 9, where the obfuscated processing happens.

11.2 For a defender hardening the CDM

The obfuscation distribution that we observe matches the design prescription of the whitebox-cryptography literature: protect the slots that ingest structured attacker input at the depth at which structured parsing begins. The depth distinction is significant. Putting MBA obfuscation at every layer of the dispatch would be wasteful and would degrade the maintainability of the binary; putting it only at the deepest layer where the actual parser logic runs is precisely where it provides the most protection per unit of obfuscation cost.

The HW-decode dormancy phenomenon itself acts as a defensive property: by routing per-frame decryption out of the public CDM interface entirely, the L1 architecture eliminates the per-frame attack surface that an in-renderer attacker would otherwise have. The trade-off is that this routing is platform-specific (it requires PMP / Media Foundation on Windows; the equivalent on macOS is FairPlay / VideoToolbox, on Android is Widevine-modulated HW decoder) and consequently the architectural claim only generalises to the Windows L1 stack as such.

11.3 For follow-on researchers

We hope two of the paper's design choices are useful to subsequent work in this area. First, the methodology is structured to be applied to any future `widevinecdm.dll` build with no manual RVA editing: the `cdm_arch_discover.py` script locates the outer vtable from the binary alone, and the runtime instrumentation script takes the discovered RVA as a parameter. Re-running the same methodology against a future Chrome stable, or against the `libwidevinecdm.so` variants that ship on Linux and Chrome OS, is mostly a matter of running the discoverer.

Second, the false-start narrative in Section 5.7 is intended to short-circuit two common dead ends that look superficially correct and that we wasted time on. Researchers who walk into this same RE space should expect to encounter several internal nineteen-slot dispatch surfaces that are *not* the public CDM_11 vtable and should expect the first-attempt obfuscation-attribution layer to be wrong by one indirection. The constructor-trace pivot from `CreateCdmInstance` is the reliable disambiguator.

12 Ethics, Disclosure, and What We Did Not Do

This study contains no:

- decrypted frame, in any form, captured to disk, sent over IPC, or retained transiently across the run of the shim;
- ciphertext byte, transient or persistent, retrieved through the shim;
- key material, content key, key wrap, or content-encryption key, captured at any layer;
- attack against `peauth.sys`, the Windows PMP, or any decoder-side component positioned to materialise a decoded frame;
- exploit code, weaponized shim, or off-the-shelf content-circumvention tool;
- production network attack against Netflix, Bitmovin, or any EME application studied;
- reverse-engineering result we cannot reproduce from the released source-only methodology against the same binary.

We have forensically verified the absence of content bytes in every artefact we produced. The captured shape logs are pure ASCII JSON totalling ∼56 MB and contain only the metadata fields described in Section 6.3. The unique `key_id` hash count in the captured runtime events is zero: the code path that would have computed a hash of a Netflix key_id was never exercised at runtime, because the slots that take an `InputBuffer_2*` argument were not invoked.

No vendor was notified prior to this submission because the work does not constitute a vulnerability report. We will coordinate with Google's Chrome Vulnerability Reward Program and the relevant Widevine security channels prior to any future submission of exploit-class findings derived from the methodology here.

13 Conclusion

We have presented a byte-verified architectural model of the modern Widevine CDM_11 dispatch as it ships in two representative Chromium-family browsers, and a runtime-confirmed empirical map of which CDM_11 slots are exercised during steady-state hardware-decoded Netflix playback. The user-facing per-frame slots are dormant under L1 hardware decode while an internal dispatch table generates ∼2.3k events per second of cryptographic or scheduler activity that is invisible from the EME boundary. The session-management slots remain reachable and exercise the same dispatch path regardless of whether the host EME application is Netflix or a non-commercial Widevine test source. The library distributes mixed-boolean-arithmetic obfuscation across the slot space in a pattern that mirrors the depth at which each slot begins to parse structured attacker-influenced data.

The work is situated in two decades of prior DRM research, follows the Buchanan model of architectural-only publication, and produces no extraction artefact. The methodology and all source needed to reproduce the empirical results are released alongside the paper.

References

[1] Google. *Widevine DRM Architecture Overview.* Google Developers documentation, accessed 2026.

[2] Chromium Project. *media/cdm/api/content_decryption_module.h.* Chromium source tree, accessed 2026.

[3] Chromium Project. *media.mojom.ContentDecryptionModule interface.* Chromium source tree, accessed 2026.

[4] Chromium Project. *Code Integrity Guard on utility processes.* Chromium documentation, accessed 2026.

[5] Microsoft Corporation. *Control Flow Guard.* Microsoft Learn documentation, accessed 2026.

[6] Microsoft Corporation. *SetProcessValidCallTargets function.* Microsoft Learn documentation, accessed 2026.

[7] W3C. *Encrypted Media Extensions, W3C Recommendation 18 September 2017.*

[8] W3C. *ISO Common Encryption EME Stream Format and Initialization Data.*

[9] ISO/IEC 23001-7. *Information technology — MPEG systems technologies — Part 7: Common encryption in ISO base media file format files.*

[10] D. Buchanan. *Personal blog:* https://www.da.vidbuchanan.co.uk/. Series of architectural reverse-engineering writeups, 2019–present.

[11] D. Buchanan. *Analysing the iMessage NaCl key-wrap.* Personal blog, 2023.

[12] D. Buchanan. *Apple T2 secure boot chain analysis.* Personal blog, 2022.

[13] A. Huang. *Hacking the Xbox: An Introduction to Reverse Engineering.* No Starch Press, 2003.

[14] A. Huang and S. Cross. *NeTV: A Stream Mediation Device, and the DMCA Exemption Process.* Project documentation, 2011–2015.

[15] *United States v. ElcomSoft Ltd., 203 F. Supp. 2d 1111 (N.D. Cal. 2002).*

[16] *Det Offentlige v. Jon Lech Johansen.* Borgarting Court of Appeal, Norway, 2003.

[17] *Universal City Studios, Inc. v. Reimerdes, 111 F. Supp. 2d 294 (S.D.N.Y. 2000).*

[18] Reporting and analysis of Israeli prosecutions of DRM reverse engineers in the early 2000s, secondary sources.

[19] Quarkslab security research team. *Reverse engineering of Widevine: a year of research.* Quarkslab technical blog, 2019.

[20] Quarkslab security research team. *Inside Widevine on Android: the OEMCrypto TrustZone trampoline.* Quarkslab technical blog, 2020.

[21] Independent contributors. *Widevine L3 decryptor: reduction of software-only Widevine to a clean-room C implementation.* GitHub, 2019–present.

[22] Independent contributors. *ANGO L3 Widevine decryptor.* GitHub, 2021–present.

[23] Check Point Research. *Various analyses of Widevine on Android.* Check Point Research blog, 2019–2024.

[24] R. Thomas et al. *LIEF: Library to Instrument Executable Formats.* Quarkslab, https://lief.re/, 2017–present.

[25] N. Eyrolles, L. Goubin, M. Videau. *Defeating MBA-based Obfuscation.* Proceedings of the 2016 ACM Workshop on Software PROtection.

[26] F. Biondi, S. Josse, A. Legay, T. Sirvent. *Effectiveness of synthesis in concolic deobfuscation.* Computers & Security, vol. 70, 2017.

[27] S. Chow, P. Eisen, H. Johnson, P.C. van Oorschot. *White-Box Cryptography and an AES Implementation.* SAC 2002.

[28] T. Lepoint, M. Rivain. *White-box cryptography: a survey.* IACR ePrint, 2014.

[29] N.A. Quynh et al. *Capstone: Next-generation disassembly framework.* Black Hat USA 2014.

[30] E. Carrera. *pefile: Portable Executable format parser.* GitHub, 2008–present.

A Per-slot dispatch table, both binaries

Slot	Method	Edge 150 RVA	Chrome 149 RVA
0	Initialize	0x1db390	0x1e0420
1	GetStatusForPolicy	0x1da7a0	0x1df8d0
2	SetServerCertificate	0x1dab10	0x1dfba0
3	CreateSessionAndGenerateRequest	0x1dad00	0x1dfdc0
4	LoadSession	0x1dad20	0x1dfde0
5	UpdateSession	0x1dad40	0x1dfe00
6	CloseSession	0x1dad60	0x1dfe20
7	RemoveSession	0x1dad80	0x1dfe40
8	TimerExpired	0x1dada0	0x1dfe60
9	Decrypt	0x1dadc0	0x1dfe80
10	InitializeAudioDecoder	0x1dade0	0x1dfea0
11	InitializeVideoDecoder	0x1dae00	0x1dfec0
12	DeinitializeDecoder	0x1dae20	0x1dfee0
13	ResetDecoder	0x1dae40	0x1dff00
14	DecryptAndDecodeFrame	0x1dae60	0x1dff20
15	DecryptAndDecodeSamples	0x1dae80	0x1dff40
16	OnPlatformChallengeResponse	0x1daea0	0x1dff60
17	OnQueryOutputProtectionStatus	0x1daec0	0x1dff80
18	OnStorageId	0x1db310	0x1e03a0

B Shape-log schema

The captured JSONL events use one of two shapes.

Generic call (for slots that are not `InputBuffer_2`-consuming):

```
{
  "kind" : "call",
  "slot" : <int 0..18>,
  "n"    : <monotonic call number for this slot>,
  "tsc"  : <QueryPerformanceCounter snapshot>,
  "rh"   : <FNV-1a-32 hash of rcx XOR rdx XOR r8 XOR r9>
}
```

Shape (for slots 9, 14, 15):

```
{
  "kind" : "ib",
  "slot" : <9|14|15>,
  "n"    : <monotonic call number>,
  "tsc"  : <QueryPerformanceCounter snapshot>,
  "es"   : <encryption_scheme; 0=clear, 1=CENC-CTR, 2=CBCS>,
  "is"   : <data_size; size only, never the bytes>,
  "kis"  : <key_id_size>,
```

```
  "kh"   : <FNV-1a-32 hash of first 16 bytes of key_id>,
  "ivs"  : <iv_size>,
  "ivnz" : <1 if IV has any non-zero byte, else 0>,
  "ns"   : <num_subsamples>,
  "pc"   : <pattern_crypt>,
  "ps"   : <pattern_skip>,
  "mt"   : <media timestamp>
}
```

Notably absent fields: `data`, `key_id` bytes, `iv` bytes, `subsamples` array, output-frame contents, content-encryption key, key-wrap key, OEMCrypto internal state. These are deliberately omitted.